Steering Through the Trees: Physical Lie Vector Field Evasion of Tree Ensembles in Network Intrusion Detection

Quang-Vinh Dang[a,*]

[a] *School of Innovative and Computing Sciences, British University Vietnam, Hung Yen, Vietnam*

Abstract

Gradient-boosted decision trees dominate flow-based network intrusion detection, yet their robustness to schema-feasible evasion is rarely measured at scale. Feature-space attacks ignore the coupling invariants of flow records; packet mutators need hundreds of queries. We propose Flow-Field Evasion (FFE), which models four packet-level actions (temporal dilation, payload padding, chaff injection, TCP window morphing) as infinitesimal generators of a Lie cone on the flow manifold, steers along the most cost-effective generator and finishes with a snap to tree split thresholds. On three full, deduplicated corpora, CSE-CIC-IDS2018 (5.0 million flows), CIC-IDS2017 (1.7 million) and UNSW-NB15 (124 thousand), FFE evades XGBoost in 99.5%, 99.2% and 57.6% of the detected attacks with 15–94 queries. On the main seed it is significantly more successful than seven baselines, including surrogate-based CAPGD (88.5%, 89.5%, 21.1%); over five retrained seeds its mean Real ASR (99.7%, 97.3%, 58.3%) stays above CAPGD's (97.7%, 82.4%,

*Corresponding author

Email address: vinh.dq4@buv.edu.vn (Quang-Vinh Dang)

31.5%). A white-box tree attack that projects every candidate matches or exceeds FFE on all three corpora (99.5%, 100.0%, 90.2%). Lie-guided adversarial training nearly halves FFE success at the budget it was trained on (84.1% to 46.5%) but protects much less against a larger budget (80.3%).
Keywords: Adversarial machine learning, Network intrusion detection, Gradient boosted decision trees, Evasion attacks, Problem-space constraints, Lie algebra, Adversarial training

1. Introduction

Modern network intrusion detection systems (NIDS) increasingly rely on machine learning over flow-based telemetry. At multi-gigabit rates, inspecting every packet payload is prohibitive, so sensors such as CICFlowMeter [1] aggregate bidirectional packet exchanges into tabular flow records of timing, length, header and flag statistics. Gradient-boosted decision trees (GBDTs), notably XGBoost [2], LightGBM [3] and CatBoost [4], are the dominant classifiers for these records.

Near-perfect accuracy on benchmark corpora does not guarantee robustness in operation. Closed-world evaluations reward models that exploit testbed artifacts [5] and shortcut features [6]. Attackers, however, control the malleable parts of their traffic, such as timing, padding and window sizes, so a detector that relies on these features can be evaded without losing the exploit [7].

Evading a tree ensemble is hard for two reasons. First, the ensemble is piecewise constant, so $\nabla_x F(x) = 0$ for almost all x and gradient-based attacks receive no signal. Second, flow features obey conservation laws. Flow

Bytes/s satisfies $x_8 = 10^6(x_4 + x_5)/x_6$, where x_4 and x_5 are the forward and backward byte totals and x_6 is the duration in microseconds. A perturbation that changes the byte totals without updating the rate produces a phantom flow that protocol sanity checks reject [8, 7].

Existing work follows one of two paradigms, and neither is adequate. Feature-space methods such as ZOO [9] and HopSkipJump [10] ignore the physical coupling: on the complete CSE-CIC-IDS2018, CIC-IDS2017 and UNSW-NB15 captures, HopSkipJump yields no feasible evasive flow at all (100.0% nominal evasion on two corpora, but 0.0% Real ASR, the share of attacks that evade and satisfy the schema invariants, on all three) and ZOO keeps at most 25.2% Real ASR. Constraint-aware search such as MOEVA, the search stage of CAA [11, 12], and packet-level swarm search such as Traffic Manipulator [13] respect more of the physics, but query the classifier as a black box, needing 183–445 queries per flow. Recent surrogate-based attacks [11, 14] are cheap at attack time but depend on how closely a surrogate matches the target.

This article proposes Flow-Field Evasion (FFE), which combines both properties. FFE derives physical perturbations as differential generators, so the continuous flow satisfies the invariants exactly and a projection enforces them on the discrete schema, and it picks the direction with score probes of the model, adding a snap to the nearest tree split when the thresholds are known. To our knowledge, no prior attack we know of derives closed-form physical generators and steers with score probes (a greedy white-box leaf-tuple-style control that projects every candidate reaches 99.5%, 100.0% and 90.2%, Section 4.2).

The main contributions are as follows.

- We model four packet-level attacker actions (temporal dilation, payload padding, chaff injection and TCP window morphing) as closed-form infinitesimal generators of a Lie cone on the flow manifold.
- We prove that trajectories in this cone preserve the rate, mean-length, header and MTU invariants of flow records in the continuous relaxation of the flow manifold.
- We present FFE, which moves each flow along the generator with the largest cost-weighted score decrease and finishes with tree-split boundary snapping (TSBS).
- We evaluate FFE at full scale on the full, deduplicated CSE-CIC-IDS2018 (5.0 million flows), CIC-IDS2017 (1.7 million) and UNSW-NB15 (124 thousand) captures against XGBoost and LightGBM, with seven baselines. FFE reaches a Real ASR of 99.5%, 99.2% and 57.6% against XGBoost with 100.0% schema feasibility and 14.6–93.7 queries per flow, significantly above all seven baselines after Holm correction. It beats surrogate-based CAPGD (88.5%, 89.5%, 21.1%) on the main seed and in 14 of 15 retrained runs (mean 99.7%, 97.3%, 58.3% against 97.7%, 82.4%, 31.5%), but CAPGD needs far fewer target queries and boundary snapping adds at most three points, so snapping explains little of FFE's success; we did not separate the Lie structure from a projected greedy search. A white-box leaf-tuple-style attack that projects every candidate matches FFE on CSE-CIC-IDS2018 and exceeds it on CIC-IDS2017 (100.0%) and UNSW-NB15 (90.2%), so FFE

without snapping (99.5%, 97.5%, 54.6%) is the best attack that needs only scores, not the strongest attack. LightGBM on CIC-IDS2017 is evaded in only 35.1% of the cohort because FFE's window target lies in an empty region of the training data (Supplementary Section S9).

- Lie-guided adversarial training nearly halves FFE success at the budget used in training (84.1% to 46.5%) but protects much less against a larger budget (98.9% to 80.3%) or CAPGD (88.3% to 74.3%); combined with hiding the malleable features it is the least evaded by those stronger attacks (62.0% and 55.1%), and no tested defense brings evasion below 46%.

2. Related Work and Problem Formulation

2.1. Related Work and State-of-the-Art Taxonomy

Gradient-boosted decision trees (GBDTs), including XGBoost [2], LightGBM [3] and CatBoost [4], are the standard choice for flow-based network intrusion detection on benchmarks such as CSE-CIC-IDS2018 [1] and UNSW-NB15 [15]. Comprehensive empirical studies support this choice [16, 17].

Early adversarial attacks on NIDS were transplanted from image classification, where Szegedy et al. [18], Goodfellow et al. [19] and Papernot et al. [20] showed that gradient-based perturbations fool deep networks, and optimizers such as the Carlini-Wagner L_2 attack [21] followed. Applied to flow features, such methods reported high success rates, but the perturbed vectors were not network flows: they assign fractional values to packet counts, modify immutable fields such as destination port, and break the conservation between byte totals and packet counts. The problem-space view corrects

this. Pierazzi et al. [8] showed that an adversarial perturbation is meaningful only when it is realized as a transformation of the physical object, such as a binary or a packet sequence, and that the transformation must preserve functionality and survive the feature extractor. Verkerken et al. [22] apply it to host-space perturbations against NIDS. Surveys and audits of this literature [23, 24, 25] argue that many attacks assume threat models that attackers cannot realize, and that success rates overstate the risk once realistic constraints are enforced [7]. Constrained tabular attacks address feasibility only in part: CAA [11] combines a gradient-based attack (CAPGD) with the MOEVA genetic search [12], which handles constraints by penalties, and Traffic Manipulator [13] mutates PCAP files with particle swarm search over intervals, padding and chaff.

Recent work makes the realism requirement concrete for NIDS. Merzouk et al. [26] show that many published evasion attacks cannot be realized on live traffic, Sheatsley et al. [27] and Debicha et al. [28, 29] craft domain-constrained or botnet flows that remain valid, Zhang et al. [30] build explainable and transferable attacks, and El Shehaby et al. [31] score which flow features an attacker can perturb, the mutable/immutable split that FFE builds into its schema. Learned generators [32, 14] trade query cost for a training phase. For tabular data in general, CAPGD and CAA [11], cost-aware attacks [33] and recent benchmarks and surveys [34, 35, 36, 37] stress the role of constraints in reported success rates. The tree-ensemble literature moved toward faster and deployable attacks [38], evasion detection [39], and robust training [40, 41, 42, 43, 44]. None combines closed-form coupled-invariant generators with score-only steering.

Evaluation practice has also been questioned: CIC-IDS2017 and CSE-CIC-IDS2018 contain labeling errors and duplicated records [45, 46, 47], and detectors generalize poorly across datasets [48]. We therefore evaluate on complete, deduplicated captures (Section 4.1).

Exact methods exploit the piecewise-constant structure of trees. Kantchelian et al. [49] cast minimal evasion against tree ensembles as a mixed-integer linear program. The formulation is exact but NP-hard and accepts only linear constraints, so it cannot express the nonlinear rate relations of flow records. MaxSAT encodings of tree ensembles [50] and SAT-based explanations of decision trees [51] provide formal explanations and, for ensembles, robustness checks, and LT-Attack [52] attacks tree ensembles through a discrete search over leaf tuples. The MILP and LT-Attack operate in unconstrained feature space and do not enforce coupled network invariants. Robust training of tree ensembles [53, 54, 55] and adversarial retraining of neural NIDS [56] improve robustness within the same feature-space threat model.

Supplementary Table S10 summarizes these paradigms and the gap they leave. Methods that exploit the structure of tree ensembles or work in unconstrained feature space do not enforce the coupled flow invariants unless a projection is added (Supplementary Section S7), while methods that preserve physical feasibility either treat the detector as a black box and need hundreds of queries per flow, or train a surrogate and transfer, which is cheap per flow but depends on how well the surrogate mimics the target. Flow-Field Evasion (FFE) addresses this gap by steering flows along infinitesimal packet-level generators that keep every step on the flow manifold, choosing the direction with score probes and, when the thresholds are known, finishing

with a snap to the nearest split.

2.2. Valid Packet Streams and Flow Telemetry

A packet stream $P = ((p_1, t_1), \ldots, (p_m, t_m))$ is an ordered sequence of packets with $0 \leq t_1 \leq \cdots \leq t_m$, where t_i is the transmission time at the network interface card. Only streams in the admissible set $\mathcal{P}_{\text{valid}}$ are considered. Every packet must carry valid IPv4, TCP and UDP checksums [57, 58, 59] and obey the interface MTU:

$$\forall i \in \{1, \ldots, m\}, \quad \text{Len}_{\text{IP}}(p_i) \leq \text{MTU}_{\text{IP}} = 1500 \text{ bytes} \tag{1}$$

A flow is the bidirectional set of packets sharing a 5-tuple, split into forward (client-to-server) and backward packets. A flow exporter such as CICFlowMeter [1] computes a deterministic, many-to-one mapping

$$\Phi : \mathcal{P}_{\text{valid}} \to \mathbb{R}^d \tag{2}$$

and $x = \Phi(P)$ summarizes the transmission.

2.3. Twelve-Dimensional Feature Schema

We use a standardized 12-dimensional schema of the core metrics that state-of-the-art models analyze on CSE-CIC-IDS2018 [1]. The feature vector $x \in \mathbb{R}^{12}$ comprises: x_0, destination port; x_1, protocol (6 for TCP, 17 for UDP); x_2, number of forward packets; x_3, number of backward packets; $x_4 = \sum_{p_i \in P_{\text{fwd}}} \text{Len}(p_i)$, forward wire bytes including IP and transport headers; x_5, backward bytes; $x_6 = (t_m - t_1) \times 10^6$, flow duration in microseconds; x_7, initial forward TCP window ($0 \leq x_7 \leq 65535$, and $x_7 \equiv 0$ for UDP); x_8, flow bytes per second; x_9, flow packets per second; x_{10}, mean forward packet

length; and x_{11}, forward transport header bytes, which must be at least the per-packet minimum header size times x_2. The rates and mean are fixed by the relation

$$x_8 = g_8(x_4, x_5, x_6) = \frac{10^6\,(x_4 + x_5)}{x_6}, \qquad x_9 = g_9(x_2, x_3, x_6) = \frac{10^6\,(x_2 + x_3)}{x_6},$$
$$x_{10} = g_{10}(x_4, x_2) = \frac{x_4}{\max(1, x_2)} \tag{3}$$

2.4. Feature Invariants

The indices $\mathcal{D} = \{0, \ldots, 11\}$ fall into five constraint classes derived from network semantics, and a feature may belong to more than one class.

- $\mathcal{I}_{\text{imm}} = \{0, 1, 3, 5\}$: port, protocol and backward packets and bytes are fixed by the service and the server, so $\Delta x_i = 0$.
- $\mathcal{I}^{+}_{\text{mono}} = \{2, 4, 6, 11\}$: an attacker may add packets, pad payloads or delay transmissions, but cannot remove bytes the payload needs, so $\Delta x_j \geq 0$.
- $\mathcal{I}_{\text{bound}} = \{7\}$: the window stays within $0 \leq x_7 + \Delta x_7 \leq 65535 \cdot \mathbb{I}_{x_1=6}$.
- $\mathcal{I}_{\text{dep}} = \{8, 9, 10\}$: these features are the deterministic functions g_8, g_9, g_{10} of the base metrics given above.
- $\mathcal{I}_{\text{ineq}} = \{4, 11\}$: $x_{11} \geq x_2(20\,\mathbb{I}_{x_1=6} + 8\,\mathbb{I}_{x_1=17})$ and $0 \leq x_4 \leq 1500 x_2$.

Feature x_4 is both monotone and MTU bounded. The set $\mathcal{I}^{+}_{\text{mono}}$ gives the mutable monotone features that Algorithm 2 modifies.

2.5. Flow Manifold and Ambient Relaxation

The physical flow space is the discrete image of valid packet streams under the telemetry exporter:

$$\mathcal{M}_{\text{flow}} = \left\{ x \in \mathbb{R}^{12} \;\middle|\; \exists P \in \mathcal{P}_{\text{valid}} \text{ s.t. } \Phi(P) = x \right\} \subset \mathbb{Z}^9_{\geq 0} \times \mathbb{R}^3_{\geq 0} \tag{4}$$

where discrete coordinates $(x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_{11})$ satisfy integrality (with packet counts $x_2 \in \mathbb{N}_{\geq 1}, x_3 \in \mathbb{N}_{\geq 0}$, duration in integer microseconds $1 \leq x_6 \leq 1.1 \times 10^8\,\mu$s bounded below the 120-second exporter timeout), rates $(x_8, x_9, x_{10}) \in \mathbb{R}^3_{\geq 0}$, and packet lengths comply with interface MTU restrictions ($x_4 \leq 1500 x_2$). We focus on IPv4 ($L_{\text{IPv4}} = 20$ bytes) and standard Ethernet MTU.

To enable continuous differential geometric optimization, we define the *continuous ambient flow manifold* $\widetilde{\mathcal{M}}_{\text{flow}} \subset \mathbb{R}^{12}$ by relaxing packet counts and duration to positive reals ($x_2 \geq 1, x_6 > 0$), with transport header limits $H_{\min}(x_1) = 20 \cdot \mathbb{I}_{x_1=6} + 8 \cdot \mathbb{I}_{x_1=17}$ and $H_{\max}(x_1) = 60 \cdot \mathbb{I}_{x_1=6} + 8 \cdot \mathbb{I}_{x_1=17}$:

$$\widetilde{\mathcal{M}}_{\text{flow}} = \left\{ x \in \mathbb{R}^{12} \;\middle|\; \begin{array}{l} x_0 \in [1, 65535], \quad x_1 \in \{6, 17\}, \quad x_2 \geq 1, \quad x_3, x_5 \geq 0, \\ H_{\min}(x_1) x_2 \leq x_{11} \leq H_{\max}(x_1) x_2, \\ 0 \leq x_4 \leq 1500 x_2, \quad 0 < x_6 \leq 1.1 \times 10^8, \\ 0 \leq x_7 \leq 65535 \cdot \mathbb{I}_{x_1=6}, \quad x_{10} = x_4 / x_2, \\ x_8 = 10^6 (x_4 + x_5)/x_6, \quad x_9 = 10^6 (x_2 + x_3)/x_6 \end{array} \right\} \tag{5}$$

Continuous vector field flows are formulated on $\widetilde{\mathcal{M}}_{\text{flow}}$, while discrete execution snaps continuous trajectories back to $\mathcal{M}_{\text{flow}}$ via the closed-form projection operator $\mathcal{P}_{\mathcal{M}}$ (Section 3.4).

2.6. Threat Model

The adversary starts from a malicious flow $x_{\text{orig}} = \Phi(P) \in \mathcal{M}_{\text{flow}}$ classified as an attack, $P(\text{Malicious} \mid x_{\text{orig}}) \geq 0.5 \iff F(x_{\text{orig}}) \geq 0$, and seeks a modified flow $x^* = \Phi(P^*) \in \mathcal{M}_{\text{flow}}$ with

$$P(\text{Malicious} \mid x^*) < 0.5 \iff F(x^*) < 0 \tag{6}$$

Tier 1 (white-box) grants access to the ensemble architecture, leaf weights and split thresholds Θ_j, which enables exact Tree-Split Boundary Snapping (TSBS) and forms the primary benchmark (Section 4.2). Tier 2 (black-box) allows only query access or zero-knowledge transfer of flows crafted on a surrogate ensemble, which Supplementary Section S5 evaluates.

The transformed trace P^* must preserve the malicious semantics, $\mathcal{S}(P^*) = \mathcal{S}(P)$, so that exploit delivery, shellcode execution, credential cracking or service degradation still succeeds on the victim. Payload padding preserves discrete payload exploits by appending non-executable, protocol-compliant bytes (SSH ignore messages, RFC 4253 [60]). Temporal dilation still delivers every packet, although it lowers the burst intensity of volumetric floods (Section 6.1); we do not test that the attack's effect on the victim is preserved.

The NIDS is an ingress biflow sensor (for example, an IPFIX collector at a border router or DMZ firewall). The adversary controls only the client endpoint, so it changes the forward features $x_2, x_4, x_6, x_7, x_{11}$ and the derived rates, while the server-generated x_3 and x_5 are immutable. Forward chaff is sent as pure ACKs with seq = SND.NXT and ack = RCV.NXT. A TCP endpoint never acknowledges a pure ACK (RFC 9293 [58], § 3.8.1), so these packets elicit no backward packets or bytes and $\Delta x_3 = \Delta x_5 = 0$.

Chaff is spaced or interleaved with forward payload and window updates so that three duplicate ACKs do not trigger fast retransmit (RFC 5681 [61], § 3.2). Window morphing edits the window field of the initial SYN consistently with window scaling (RFC 7323 [62]). For encrypted protocols such as SSH, padding is inserted at the application layer before cryptographic encapsulation (RFC 4253 [60], § 11.2).

The adversary minimizes the total operational cost

$$\min \quad \mathcal{C}(x^*, x_{\text{orig}}) = \sum_{k=1}^{4} c_k s_k \tag{7}$$

where s_k is the amount of generator k applied and c_k is its unit cost in transmission latency, bandwidth expansion and socket buffer allocation.

3. The Flow-Field Evasion (FFE) Framework

Figure 1 summarizes the framework.

3.1. Geometry of the Flow Manifold and Its Generators

The physical flow set $\mathcal{M}_{\text{flow}} = \Phi(\mathcal{P}_{\text{valid}})$ is a discrete lattice in $\mathbb{R}^{12}$. Its differential geometry is analyzed through the smooth relaxation $\widetilde{\mathcal{M}}_{\text{flow}}$ (Section 2.2), defined as the zero set of the conservation residual Ψ below, subject to $x_2 \geq 1$, $0 < x_6 \leq 1.1 \times 10^8$, $x_{11} \geq x_2(20 \cdot \mathbb{I}_{x_1=6} + 8 \cdot \mathbb{I}_{x_1=17})$ and $0 \leq x_4 \leq 1500 x_2$.

$$\Psi(x) = \begin{bmatrix} x_8 - \dfrac{10^6(x_4 + x_5)}{x_6} \\ x_9 - \dfrac{10^6(x_2 + x_3)}{x_6} \\ x_{10} - \dfrac{x_4}{x_2} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \tag{8}$$

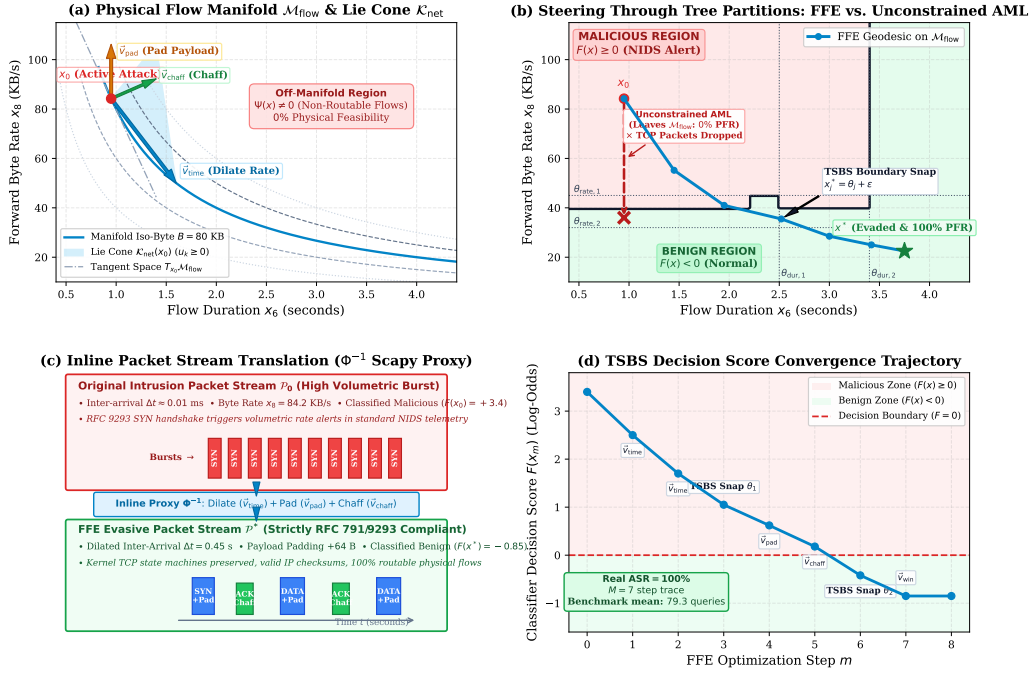


Figure 1: Architecture of the Flow-Field Evasion (FFE) framework. **(a)** The physical conservation constraints define the flow manifold $\mathcal{M}_{\text{flow}}$, and the generators span a forward Lie cone that keeps trajectories on it. **(b)** Axis-aligned tree partitions push unconstrained feature-space perturbations off the manifold, whereas FFE moves along the manifold and finishes with tree-split boundary snapping (TSBS). **(c)** Packet-level realization of the flow transformations through an inline proxy. **(d)** Decision score descent over the steps of an attack trace.

By the Regular Value Theorem, at any point where the Jacobian $J_\Psi(x) \in \mathbb{R}^{3\times 12}$ has full rank 3, the tangent space is $T_x\widetilde{\mathcal{M}}_{\text{flow}} = \ker J_\Psi(x)$. A physically realizable velocity $\vec{v} = [\dot{x}_0, \ldots, \dot{x}_{11}]^T$ therefore satisfies the linearized

conservation laws:

$$\dot{x}_8 = \frac{10^6(\dot{x}_4 + \dot{x}_5)}{x_6} - \frac{x_8}{x_6}\dot{x}_6 \tag{9}$$

$$\dot{x}_9 = \frac{10^6(\dot{x}_2 + \dot{x}_3)}{x_6} - \frac{x_9}{x_6}\dot{x}_6 \tag{10}$$

$$\dot{x}_{10} = \frac{\dot{x}_4 - x_{10}\dot{x}_2}{x_2} \tag{11}$$

An adversary controls four elementary degrees of freedom, each generating a 1-parameter flow whose push-forward is a tangent vector field. Here $\vec{e}_k$ is the kth standard basis vector of $\mathbb{R}^{12}$ (indexed from 0):

$$\begin{aligned}
\vec{v}_{\text{time}} &= \vec{e}_6 - \frac{x_8}{x_6}\vec{e}_8 - \frac{x_9}{x_6}\vec{e}_9, \\
\vec{v}_{\text{pad}} &= \vec{e}_4 + \frac{10^6}{x_6}\vec{e}_8 + \frac{1}{x_2}\vec{e}_{10}, \\
\vec{v}_{\text{chaff}} &= \vec{e}_2 + L_0\vec{e}_4 + \frac{L_0 \cdot 10^6}{x_6}\vec{e}_8 + \frac{10^6}{x_6}\vec{e}_9 + \frac{L_0 - x_{10}}{x_2}\vec{e}_{10} + H_0\vec{e}_{11}, \\
\vec{v}_{\text{win}} &= \mathbb{I}_{x_1=6}\ \text{sat}\left(\frac{W_{\text{target}} - x_7}{\epsilon_{\text{win}}}\right)\vec{e}_7,
\end{aligned} \tag{12}$$

where temporal dilation adds delay without bytes or packets, payload padding adds forward dummy bytes, chaff injection adds forward zero-payload probes, and window morphing moves the client window toward W_{target}. Here $\text{sat}(z) = \max(-1, \min(1, z))$ and $\epsilon_{\text{win}} > 0$. The probe sizes are $L_0 = 40$, $H_0 = 20$ for empty TCP segments per RFC 9293 §3.8.4 [58], $L_0 = 52$, $H_0 = 32$ with TCP timestamps [62], and $L_0 = 28$, $H_0 = 8$ for UDP [59]. Probes are classified at the ingress perimeter before server ACKs alter backward counts; bidirectional sessions are discussed in Section 6.1. The window field is globally Lipschitz rather than C^∞ and depends only on x_1 and x_7. The implementation (Supplementary Algorithm S1) uses its $\epsilon_{\text{win}} \to 0$ limit, $\mathbb{I}_{x_1=6}\,\text{sign}(W_{\text{target}} - x_7)\,\vec{e}_7$, which moves by $|W_{\text{target}} - x_7|$ and lands exactly on W_{target} without overshoot.

3.2. Commutativity, Invariants and the Lie Cone

The Lie bracket of vector fields is $[X, Y] = J_Y X - J_X Y$, where J_X and J_Y are their Jacobians.

Proposition 1 (Commutativity and Integrability of Physical Flow Generators)**.** *Because protocol $x_1 \in \{6, 17\}$ is an immutable attribute ($\dot{x}_1 \equiv 0$), the ambient manifold $\widetilde{\mathcal{M}}_{flow}$ decomposes into two disjoint smooth invariant sub-manifolds: $\widetilde{\mathcal{M}}^{(6)}_{flow}$ (TCP flows) and $\widetilde{\mathcal{M}}^{(17)}_{flow}$ (UDP flows). On each component, the indicator $\mathbb{I}_{x_1=6}$ is constant, and L_0, H_0 are constant along any trajectory (the implementation selects $H_0 = 32$ iff the mean forward header $x_{11}/x_2 \geq 26$; injecting chaff of header size H_0 moves that mean toward H_0 and never across the threshold, so the selection is invariant under the flow), and denominators satisfy $x_2 \geq 1, x_6 > 0$, ensuring $\vec{v}_{time}, \vec{v}_{pad}, \vec{v}_{chaff}$ are C^∞ vector fields and $\vec{v}_{win}$ is globally Lipschitz (its brackets exist almost everywhere and vanish there, because x_7 is neither read nor written by the other three generators). The Lie brackets between all four infinitesimal packet transformation generators vanish identically on $\widetilde{\mathcal{M}}_{flow}$:*

$$[\vec{v}_i, \vec{v}_j] \equiv \mathbf{0} \in \mathbb{R}^{12}, \quad \forall i, j \in \{1, 2, 3, 4\} \tag{13}$$

Consequently, the generators span an Abelian Lie algebra $\mathfrak{g}_{flow} \cong \mathbb{R}^4$ (or $\mathbb{R}^3$ for UDP).

The proof of Proposition 1, and the Frobenius integrability corollary that follows from it (Supplementary Corollary S1), are given in the Supplementary Material (Section S1).

The admissible deformation cone is the positive span of the generators:

$$\mathcal{K}_{\text{net}}(x) = \left\{ \sum_{k=1}^{4} u_k \vec{v}_k(x) \;\middle|\; u_k \geq 0 \right\} \subset T_x \widetilde{\mathcal{M}}_{\text{flow}} \tag{14}$$

Theorem 1 (Universal Invariant Preservation)**.** *Let $x_0 \in \widetilde{\mathcal{M}}_{flow}$ be an initial network flow vector. For any measurable and locally bounded control signal $u(t) \in L^\infty_{loc}(\mathbb{R}_{\geq 0}, \mathbb{R}^4_{\geq 0})$ satisfying the admissible boundary rules ($u_2(t) \leq (1500 - L_0)u_3(t)$ whenever $x_4(t) = 1500x_2(t)$, and $u_1(t) = 0$ whenever $x_6(t) = 1.1 \times 10^8$), the trajectory $x(t)$ defined by the dynamical system:*

$$\dot{x}(t) = \sum_{k=1}^{4} u_k(t)\vec{v}_k(x(t)), \quad x(0) = x_0 \tag{15}$$

satisfies $x(t) \in \widetilde{\mathcal{M}}_{flow}$ for all $t \geq 0$, with solutions existing uniquely and globally on $[0, \infty)$. Under discrete stepping, the continuous Lie cone guides directional descent while the closed-form analytical projection operator $\mathcal{P}_{\mathcal{M}}$ (Section 3.4) eliminates discrete $\mathcal{O}(\Delta^2)$ truncation drift and enforces packet and byte integrality ($x_2 \in \mathbb{N}_{\geq 1}, x_4, x_6 \in \mathbb{N}$) and MTU clamping $x_4 \leq 1500x_2$, ensuring that the discrete state sequence remains strictly in the physical packet manifold $\mathcal{M}_{flow}$.

Proof sketch. Differentiating Ψ along the flow gives a linear homogeneous system $\dot{\Psi} = A(t)\Psi$ with diagonal, locally integrable $A(t)$, so $\Psi \equiv \mathbf{0}$ by Carathéodory's theorem [63]. The bounds $g_i \geq 0$ are affine, so the Nagumo–Brezis condition [64, 65] holds on every face and the manifold is forward invariant. The same theorem gives existence and uniqueness of global solutions. The full proof is in the Supplementary Material (Section S1).

3.3. Operational Cost and Optimal Control

The unit costs are $c_1 = 10^{-4}$ per second of induced delay, $c_2 = 10^{-2}$ per byte of padding, $c_3 = 1.0$ per probe packet, and $c_4 = 10^{-3}$ per normalized window displacement $s_4 = |W_{\text{target}} - x_7|/65535 \in [0, 1]$. The evasion problem

is an optimal control problem:

$$\min_{u(t)\geq 0} \quad \int_0^{T^*} \left(\sum_{k=1}^{4} c_k u_k(t) \right) dt \tag{16}$$

$$\text{subject to} \quad \dot{x}(t) = \sum_{k=1}^{4} u_k(t) \vec{v}_k(x(t)), \quad x(0) = x_{\text{orig}} \tag{17}$$

$$F(x(T^*)) < 0 \quad \text{(Classifier Misclassification)} \tag{18}$$

Because the generators commute (Proposition 1), the displacement depends only on $s_k = \int_0^{T^*} u_k(t)\, dt \geq 0$, and the cost $\mathcal{C}(s) = \sum_{k=1}^{4} c_k s_k$ is linear and path-independent. The tree ensemble boundary $F(x) = 0$ is piecewise constant and non-convex, so FFE greedily follows the generator with the largest directional score decrease per unit cost, $-\nabla_k / c_k$, and finishes with boundary snapping (TSBS).

3.4. Algorithmic Realization of Flow-Field Evasion

Algorithm 1 gives the master loop of Flow-Field Evasion (FFE). Starting from a flagged flow $x_0 \in \mathcal{M}_{\text{flow}}$ with $F(x_0) \geq 0$, each iteration probes the four generators, takes the admissible step with the largest score reduction per unit cost, and falls back to boundary snapping or temporal dilation when no generator lowers the score.

Supplementary Algorithm S1 estimates the directional derivatives by probing the ensemble once per generator, since ∇F vanishes almost everywhere. The probe steps are $\Delta s_1 = \max(1000,\, 0.5x_6)$ μs, $\Delta s_2 = \max(50,\, 0.25x_4)$ bytes, $\Delta s_3 = \max(1,\, \text{round}(0.5x_2))$ packets and $\Delta s_4 = |64240 - x_7|$, and the derivatives are taken per second of delay, per byte, per packet and per window unit of $\Delta s_4/65{,}535$, respectively. Every displacement is followed by

Algorithm 1 Flow-Field Evasion (FFE) Master Optimization Loop

Require: Classifier F, initial flow $x_0 \in \mathcal{M}_{\text{flow}}$, schema $\mathcal{S}$, step budget M, costs $\vec{c} = [c_1, \ldots, c_4]^T$, threshold $\tau = 0$
Ensure: Evasive flow $x^* \in \mathcal{M}_{\text{flow}}$ with $F(x^*) < \tau$, or FAILURE

$x \leftarrow \mathcal{P}_{\mathcal{M}}(x_0)$; $f \leftarrow F(x)$; $q \leftarrow 1$
for step $= 1$ **to** M **do**
  if $f < \tau$ **then**
    return $(x, \text{SUCCESS})$
  end if
  $\mathcal{D}, q_{\text{used}} \leftarrow \text{EstimateLieDerivatives}(F, x, \mathcal{S})$; $q \leftarrow q + q_{\text{used}}$ {Supplementary Algorithm S1}
  $k^* \leftarrow$ first maximizer of $(-\nabla_k)/c_k$ over $(k, \nabla_k, \vec{v}_k, \Delta s_k, f_k) \in \mathcal{D}$ with $\nabla_k < 0$; null if none
  if $k^* \neq$ null **then**
    $x \leftarrow \mathcal{P}_{\mathcal{M}}(x + \Delta s_{k^*}\, \vec{v}_{k^*})$; $f \leftarrow f_{k^*}$ {Cached probe, no extra query}
  else
    $x_{\text{snap}} \leftarrow \text{TSBS}(F, x, \mathcal{S})$
    if $F(x_{\text{snap}}) < f$ **then**
      $x \leftarrow x_{\text{snap}}$; $f \leftarrow F(x_{\text{snap}})$
    else
      $\Delta t \leftarrow \min(1.1 \times 10^8, \max(5 \times 10^4, 1.5\, x_6))$ {Fallback dilation; projection clamps duration at the 1.1×10^8 μs timeout}
      $x \leftarrow \mathcal{P}_{\mathcal{M}}(x + \Delta t\, \vec{v}_{\text{time}}(x))$; $f \leftarrow F(x)$; $q \leftarrow q + 1$
    end if
  end if
end for
if $f < \tau$ **then**
  return $(x, \text{SUCCESS})$
else
  return $(x, \text{FAILURE})$
end if

the closed-form projection $\mathcal{P}_{\mathcal{M}}$ of Supplementary Algorithm S2, which restores immutable features, enforces monotonicity and integer counts, bounds header and packet lengths, and recomputes the coupled rates.

3.4.1. Tree-Split Boundary Snapping (TSBS)

Algorithm 2 implements Tree-Split Boundary Snapping (TSBS), whose candidates are generated by Supplementary Algorithm S3. Because the score

is piecewise constant in each split feature, a small displacement cannot cross a split threshold. In Tier 1 white-box settings, TSBS therefore extracts the active split thresholds of the ensemble, keeps per feature the threshold closest to the benign side of the current value, and retains the projected candidate that most decreases F.

GBDTs also split on coupled telemetry features, so TSBS inverts their constraints in closed form. To cross an active byte-rate threshold θ_8 or packet-rate threshold θ_9, it lengthens the flow duration to

$$x_6^* = \frac{10^6(x_4 + x_5)}{\theta_8} + \epsilon_6 \quad \text{or} \quad x_6^* = \frac{10^6(x_2 + x_3)}{\theta_9} + \epsilon_6, \tag{19}$$

with $\epsilon_6 = 10\,\mu$s; the candidate is discarded if x_6^* exceeds the $1.1 \times 10^8\,\mu$s cap. To cross a forward mean packet length threshold θ_{10} from above, TSBS inverts $x_{10} = (x_4 + NL_0)/(x_2 + N)$ and adds the smallest integer chaff count that satisfies the inequality,

$$N_{\text{chaff}} = \left\lfloor \frac{x_4 - \theta_{10} x_2}{\theta_{10} - L_0} \right\rfloor + 1 \qquad (L_0 < \theta_{10} < x_{10}). \tag{20}$$

For TCP flows, TSBS also snaps the window advertisement toward the benign OS baseline $x_7^* = 64{,}240$ and toward adjacent split partitions.

3.4.2. Query Accounting

Executing all $M_{\max} = 20$ iterations without early termination costs at most $Q_{\max} \leq 4M_{\max} + 1 = 81$ model queries for the baseline query and Lie-derivative probes, or ≤ 101 if stagnation fallbacks fire, plus the TSBS candidate evaluations, which are not covered by this bound; UDP flows and TCP flows whose window already equals the benign value skip window morphing and cost $q = 3$ probes per iteration. In practice, the query count

Algorithm 2 Tree-Split Boundary Snapping (TSBS)

Require: Classifier F, flow candidate $x \in \mathcal{M}_{\text{flow}}$ with $F(x) \geq \tau$ (still flagged malicious), feature schema $\mathcal{S}$, tolerance ϵ_{rel}
Ensure: Snapped flow $x^* \in \mathcal{M}_{\text{flow}}$

$x^* \leftarrow \text{copy}(x)$
active_splits $\leftarrow$ ExtractActiveSplits(F, x^*)
$\mathcal{C} \leftarrow$ GenerateCandidates$(x^*, \text{active_splits}, \epsilon_{\text{rel}})$ {Boundary-crossing pool (Supplementary Algorithm S3)}
{Evaluate projected candidates and retain best partition crossing}
$x_{\text{best}} \leftarrow x^*$; $s_{\text{best}} \leftarrow F(x^*)$
for each $x_c \in \mathcal{C}$ **do**
    $x_{\text{proj}} \leftarrow$ ProjectManifoldInvariants$(x_c, \mathcal{S})$
    if $F(x_{\text{proj}}) < s_{\text{best}}$ **then**
        $s_{\text{best}} \leftarrow F(x_{\text{proj}})$; $x_{\text{best}} \leftarrow x_{\text{proj}}$
        if $s_{\text{best}} < 0$ **then**
            break {Early exit once benign classification is achieved}
        end if
    end if
end for
return x_{best}

depends on how close the flows sit to a split boundary that a forward generator can cross: on the full benchmarks the mean is 16.5, 14.6 and 93.7 queries per flow against XGBoost on CSE-CIC-IDS2018, CIC-IDS2017 and UNSW-NB15 (Table 1), and 14.4 queries on the balanced ablation cohort. We count every probe and every TSBS candidate evaluation as a query to the target, although a Tier 1 adversary could evaluate the candidate snaps offline on its local copy. The score probes themselves need no threshold: FFE without TSBS (Section 4.4) queries only the soft score of the target. Because $K \leq 4$ and $M \leq 20$ are constants, each iteration costs $\mathcal{O}(T \cdot D)$ tree evaluations, and Supplementary Table S10 compares this asymptotic cost with exact MILP and heuristic search.

4. Experiments and Results

4.1. Experimental Setup and Benchmark Corpora

Single-testbed evaluations have limited external validity [48], and attack evaluations that ignore realism overstate the risk [7, 23, 24, 25], so we use three widely studied corpora in full: CSE-CIC-IDS2018 [1] (nine daily files, seven of them capped at 1,048,575 rows in the distribution), CIC-IDS2017 [1] (five daily files) and UNSW-NB15 [15] (the official training and testing partition, 257,673 of the 2.54 million records of the complete capture). Every record is read and mapped onto the 12-feature schema of Section 2.2.

Cleaning is identical for all corpora. We drop records with missing or infinite values, non-positive duration, no forward packet or a protocol other than TCP or UDP, and TCP flows without an observed window; the window of UDP flows, recorded as -1 by CICFlowMeter, is set to 0. In CIC-IDS2017 we remove the 11,974 "Attempted" probes and count "Infiltration – Portscan" as port scanning; in UNSW-NB15 we drop zero-duration flows. Every flow is projected onto the flow manifold (Section 2.2), which changes a feature by more than 5% in 3.1% of the valid CSE-CIC-IDS2018 records, 4.7% of CIC-IDS2017 and none of UNSW-NB15. Records with identical features and label are then collapsed so that no flow appears on both sides of the split, a known source of inflated accuracy [45, 46, 47]: this removes 39.1%, 20.5% and 42.7% of the valid records and leaves 4,968,389, 1,658,869 and 123,508 flows (Supplementary Table S5). Records with identical features but different labels (149,526, 22 and 1,040) are kept as real ambiguity of the schema. The split into training, validation (unused) and test partitions is 70%/10%/20%, stratified by label and family, random, and not grouped by time or host, so

near-duplicates may fall on both sides and accuracies are optimistic.

The targets are XGBoost (CUDA, trained on the whole training partition) and LightGBM, each with 60 trees of depth 6 and learning rate 0.1. The methods differ in what the attacker knows. The full FFE reads the split thresholds for its snapping step (Tier 1), and we count every probe and snapping candidate as a query although a white-box attacker could evaluate its local model for free; FFE without snapping uses only the soft score, as do CAPGD and ZOO, whereas HopSkipJump uses only decisions. *ZOO-NIDS* [9], *HopSkipJump-NIDS* [10] (30 iterations, 47 queries), *MOEVA* [11, 12] (the search stage of CAA; population 25, 20 generations) and *PSO* [13] (feature-space swarm over interval and padding knobs, 15 particles, 25 iterations) only query the target. *CAPGD* [11] and the *NIDSGAN-style generator* [14] are our re-implementations: the attacker trains its own MLP (two hidden layers of 128 units) on the training flows, takes gradients on it, and queries the target once per iteration or draw (at most 20 and 5); surrogate computations are not counted. The *naive* attack is coordinate descent in $\mathbb{R}^d$. The naive attack, ZOO and HopSkipJump are unconstrained by design, MOEVA penalizes violations without repairing them, and PSO (with the closed-form projection that we added), CAPGD and the generator repair every candidate onto the manifold. Baselines use the default budgets above. The white-box tree attacks of Zhang et al. [52] and Cascioli et al. [38] need the tree structure and do not model flow constraints; the official LT-Attack code needs a Bazel and C++ build that we did not set up, so as a control we wrote a greedy leaf-tuple-style attack on the XGBoost trees, run with a projection after the attack and with a projection of every candidate (Sup-

plementary Section S7). It is not LT-Attack, and we did not run Cascioli et al.

ASR is the fraction of attacks with $P(\text{Malicious} \mid x^*) < 0.5$; PFR is the fraction of the evasive candidates that satisfy the schema invariants $x^* \in \mathcal{M}_{\text{flow}}$; Real ASR counts an attack only when it both evades and is feasible. Feasibility is checked against the invariants that FFE's projection enforces, so for FFE it holds by construction and means conformance to the schema; a synthetic packet replay is reported in Supplementary Section S8. We report Wilson 95% intervals and exact McNemar tests on paired outcomes with Holm correction over the 21 comparisons. Cost is the median over a method's own feasible evasions of the operational cost of Section 3.3 and of the feature-normalized distance $D_{L_1} = \frac{1}{d}\sum_j |x^*_j - x_j|/\sigma_j$, both computed identically for every method; the medians cover different flows and are not a paired comparison.

Cohorts are drawn once, with a fixed seed, from the malicious test flows that both XGBoost and LightGBM detect when clean (77.6%, 99.8% and 87.0% of the test attacks of CSE-CIC-IDS2018, CIC-IDS2017 and UNSW-NB15), so all results are conditional on detection. Every method except MO-EVA and PSO attacks the same 1,000 flows; those two need 183–445 queries per flow and attack the first 200 of them, and the paired tests against them use these 200. The main results use one training seed; Section 4.4 repeats the comparison for five seeds. The ablation, transferability and defense studies use a balanced cohort of up to 50 detected flows per family of CSE-CIC-IDS2018.

Table 1: Evasion benchmark against CUDA XGBoost on the full CSE-CIC-IDS2018, CIC-IDS2017 and UNSW-NB15 captures. All methods except MOEVA and PSO attack the same 1,000 detected malicious test flows; MOEVA and PSO attack the first 200 of them. ASR, PFR and Real ASR in %; PFR is the share of evasive candidates that are schema-feasible; Op. cost and D_{L_1} are medians over the method's own feasible evasions (Section 4.1); – means no feasible evasion.

Method	ASR	PFR	Real ASR [95% CI]	Queries	Op. cost	D_{L_1}	ms/flow
CSE-CIC-IDS2018 (XGBoost accuracy 97.30%, test attacks 118,963, detected by both models 92,275)							
Unconstrained feature space	36.8	88.0	32.4 [29.6, 35.4]	66.8	0.000	0.000	48.0
ZOO-NIDS [9]	30.3	83.2	25.2 [22.6, 28.0]	116.7	0.000	0.000	91.0
HopSkipJump-NIDS [10]	100.0	0.0	0.0 [0.0, 0.4]	47.0	–	–	30.3
MOEVA (search stage of CAA) [11, 12]	40.5	70.4	28.5 [22.7, 35.1]	330.1	0.092	0.001	253.2
PSO (Traffic Manipulator, projected) [13]	54.0	100.0	54.0 [47.1, 60.8]	182.9	16.3	0.061	142.0
CAPGD (surrogate) [11]	88.5	100.0	88.5 [86.4, 90.3]	3.4	46.1	0.824	15.4
NIDSGAN-style [14]	61.3	100.0	61.3 [58.2, 64.3]	2.5	48.0	0.836	7.1
FFE (ours)	99.5	100.0	**99.5 [98.8, 99.8]**	16.5	0.001	0.116	15.8
CIC-IDS2017 (XGBoost accuracy 99.96%, test attacks 79,281, detected by both models 79,150)							
Unconstrained feature space	64.8	34.4	22.3 [19.8, 25.0]	37.6	0.000	0.000	30.6
ZOO-NIDS [9]	62.2	35.0	21.8 [19.4, 24.5]	79.9	0.000	0.000	58.5
HopSkipJump-NIDS [10]	100.0	0.0	0.0 [0.0, 0.4]	47.0	–	–	36.5
MOEVA (search stage of CAA) [11, 12]	64.0	59.4	38.0 [31.6, 44.9]	225.9	0.059	0.005	201.0
PSO (Traffic Manipulator, projected) [13]	35.0	100.0	35.0 [28.7, 41.8]	251.4	15.2	0.137	203.3
CAPGD (surrogate) [11]	89.5	100.0	89.5 [87.4, 91.3]	3.2	15.0	0.305	15.1
NIDSGAN-style [14]	65.3	100.0	65.3 [62.3, 68.2]	2.4	24.6	0.490	5.8
FFE (ours)	99.2	100.0	**99.2 [98.4, 99.6]**	14.6	0.001	0.289	10.7
UNSW-NB15 (XGBoost accuracy 88.51%, test attacks 11,113, detected by both models 9,673)							
Unconstrained feature space	21.6	0.9	0.2 [0.1, 0.7]	48.5	0.082	0.000	49.1
ZOO-NIDS [9]	20.6	0.0	0.0 [0.0, 0.4]	130.1	–	–	96.5
HopSkipJump-NIDS [10]	8.1	0.0	0.0 [0.0, 0.4]	47.0	–	–	40.6
MOEVA (search stage of CAA) [11, 12]	13.0	100.0	13.0 [9.0, 18.4]	445.0	0.092	0.002	339.2
PSO (Traffic Manipulator, projected) [13]	42.0	100.0	42.0 [35.4, 48.9]	228.4	21.6	0.060	164.3
CAPGD (surrogate) [11]	21.1	100.0	21.1 [18.7, 23.7]	16.2	53.2	1.158	73.4
NIDSGAN-style [14]	11.3	100.0	11.3 [9.5, 13.4]	4.5	34.0	1.211	11.2
FFE (ours)	57.6	100.0	**57.6 [54.5, 60.6]**	93.7	5.610	0.027	89.1

4.2. Master Comparative Benchmark

Table 1 compares FFE with seven baselines on the three full-scale corpora. Evading the model in feature space is not the same as producing a feasible flow, but the gap differs by method. HopSkipJump-NIDS reaches 100.0% ASR on CSE-CIC-IDS2018 and CIC-IDS2017 yet none of its evasive candidates is feasible. The naive attack and ZOO-NIDS keep only 34.4% and 35.0% (CIC-IDS2017) and 0.9% and 0.0% (UNSW-NB15) of their evasions as feasible, but 88.0% and 83.2% on CSE-CIC-IDS2018, at a median

operational cost of essentially zero; this is consistent with moving only the coupled rate features within the 5% tolerance of the consistency check, which would mean that the check is not exact and flatters these baselines. MOEVA, which penalizes but does not repair violations, ends with a feasible share of 59.4%–100.0%.

FFE exceeds each of the seven baselines in Real ASR on all three corpora: 99.5%, 99.2% and 57.6%. All 21 comparisons with the baselines are significant after Holm correction (largest adjusted exact McNemar $p = 3.7 \times 10^{-11}$). The margin over the best baseline is 11.0 points on CSE-CIC-IDS2018 (CAPGD, 88.5%; the 110 flows that only FFE evades against none that only CAPGD evades), 9.7 points on CIC-IDS2017 (CAPGD, 89.5%; 97 against none) and, on UNSW-NB15, 22 points over PSO on the 200 flows both attacked (64.0% against 42.0%; 47 against 3), where CAPGD reaches only 21.1%. CAPGD's surrogate agrees with the target on 99.8% and 99.9% of the test flows of CSE-CIC-IDS2018 and CIC-IDS2017 but on only 87.6% of UNSW-NB15 (Table 2). A greedy white-box leaf-tuple-style control that sees the trees reaches only 31.6%, 35.1% and 0.0% when it projects after the attack, but 99.5%, 100.0% and 90.2% when every candidate is projected (Supplementary Section S7). The latter equals FFE on CSE-CIC-IDS2018, evades eight flows that FFE misses on CIC-IDS2017 (none the other way, $p = 0.008$) and gains 32 points on UNSW-NB15, at a median operational cost of 213 against 5.6. It counts two queries and reads the trees for free, whereas FFE counts every probe. FFE without snapping is the best attack that needs only scores, not the best attack.

The methods trade target queries, knowledge and cost differently. CAPGD

and the NIDSGAN-style generator need far fewer target queries than FFE (3.2–16.2 and 2.4–4.5 against 14.6–93.7), since their gradient work runs on a pre-trained surrogate that is not counted. FFE needs no surrogate; its snapping step reads the split thresholds, but the score-only variant of Section 4.4 does not. On the same 200 flows FFE needs 2.8–20 times fewer queries than PSO and MOEVA, although query counts are not strictly like-for-like across methods that use scores, decisions or a surrogate. The median operational cost of FFE's feasible evasions is 0.001, 0.001 and 5.6 units, against 15.2–21.6 for PSO, 15.0–53.2 for CAPGD and 24.6–48.0 for the generator. This is partly by design, since FFE steers by this cost, and it is specific to this cost model: in the feature-normalized distance FFE (0.116, 0.289, 0.027) is not smaller than PSO (0.061, 0.137, 0.060) on two of the three corpora.

4.3. Cross-dataset and cross-model validation

Table 2: FFE against XGBoost and LightGBM on the full corpora: clean test performance of the target, FFE success on the 1,000-flow cohort (Wilson 95% intervals), and agreement between the surrogate and the target on the test flows.

Dataset	Model	Acc. (%)	F1 (%)	AUC (%)	FFE Real ASR (%)	PFR (%)	Queries	Surr. agr. (%)
CSE-CIC-IDS2018	XGBoost	97.30	87.31	96.81	99.5 [98.8, 99.8]	100.0	16.5	99.8
	LightGBM	97.28	87.28	96.77	99.5 [98.8, 99.8]	100.0	20.2	99.7
CIC-IDS2017	XGBoost	99.96	99.91	100.00	99.2 [98.4, 99.6]	100.0	14.6	99.9
	LightGBM	99.96	99.92	100.00	35.1 [32.2, 38.1]	100.0	120.1	99.9
UNSW-NB15	XGBoost	88.51	87.32	96.47	57.6 [54.5, 60.6]	100.0	93.7	87.6
	LightGBM	88.47	87.28	96.43	62.4 [59.4, 65.3]	100.0	89.0	87.4

Table 2 shows that clean accuracy is 97.30% on CSE-CIC-IDS2018 (F1 87.3%, since 12.0% of the flows are attacks and the rare families are hard), 99.96% on CIC-IDS2017 and 88.51% on UNSW-NB15, where the 12-feature mapping is lossy and 13.0% of the test attacks are missed by at least one of the two models. On CSE-CIC-IDS2018 the XGBoost model places the root split of 51 of its 60 trees on the destination port, an immutable feature,

and is nevertheless evaded in 99.5% of the attacked flows, so immutability of the root feature does not by itself protect a model. On CIC-IDS2017 FFE succeeds far less often against LightGBM than against XGBoost (35.1% against 99.2%, with 120.1 against 14.6 queries). Of the XGBoost trees there, 36 of 60 split first on the initial window, a feature that FFE can move, 14 on the protocol and 10 on backward payload bytes; the cause is not a more robust model but the window target: FFE morphs the window to 64,240 B, a value absent from the CIC-IDS2017 training flows, and the two ensembles place their splits differently in that empty interval. Moving the target to the in-support 65,535 B lifts LightGBM from 35.1% to 93.7% and leaves XGBoost at 99.2% (Supplementary Section S9). Per-family results are in Supplementary Table S1: on UNSW-NB15 exploits, more than half of the cohort, are evaded in only 44.6% of 504 flows and set the aggregate figure.

4.4. Retraining variance, ablations and transferability

Retraining variance.. The single-seed benchmark cannot show how much of the ranking is luck of the split and the surrogate. We therefore repeated the whole pipeline for five seeds (new split, new XGBoost model, new surrogate and generator, new cohort of 300 detected flows) for FFE, CAPGD and the NIDSGAN-style generator (Table 3). FFE is at least as successful as CAPGD in all 15 runs and strictly better in 14 (the exception is a tie at 99.3% on one CSE-CIC-IDS2018 seed). The gap on CSE-CIC-IDS2018 shrinks from 11.0 points on the main seed to 2.0 points on average (99.7% against 97.7%). On CIC-IDS2017 FFE also varies less (standard deviation 1.5 points against 11.9 for CAPGD and 15.7 for the generator), possibly because the surrogate attacks depend on the surrogate fitted in that run; on UNSW-NB15 FFE

varies more than CAPGD (4.3 against 2.8).

Table 3: Real ASR (%, mean ± standard deviation over five seeds) against XGBoost; each seed retrains the target and the surrogate and draws a new cohort of 300 detected flows.

Dataset	**Acc. (%)**	**FFE**	**CAPGD**	**NIDSGAN-style**
CSE-CIC-IDS2018 (5 seeds)	97.30 ± 0.00	99.7 ± 0.3	97.7 ± 2.4	97.2 ± 2.2
CIC-IDS2017 (5 seeds)	99.96 ± 0.00	97.3 ± 1.5	82.4 ± 11.9	84.8 ± 15.7
UNSW-NB15 (5 seeds)	88.54 ± 0.23	58.3 ± 4.3	31.5 ± 2.8	24.0 ± 16.2

Role of tree-split boundary snapping.. Switching TSBS off (Supplementary Table S2) leaves Real ASR unchanged on CSE-CIC-IDS2018 (99.5% in both cases) and lowers it from 99.2% to 97.5% on CIC-IDS2017 and from 57.6% to 54.6% on UNSW-NB15, while the mean number of queries rises from 11.6, 13.5 and 51.5 to 16.5, 14.6 and 93.7 with TSBS. Snapping therefore rescues at most three more points of Real ASR and costs up to 1.8 times as many queries. TSBS therefore accounts for little of FFE's success, and we do not claim that boundary snapping makes the attack query-efficient; we did not isolate the benefit of the Lie structure from a greedy search over the same four moves.

Other ablations.. Step-budget, generator, cost-regime and snapping-offset ablations are in Supplementary Section S4: success rises from 42.8% at $M = 1$ to 98.9% at $M \geq 10$, no single generator suffices (10.2%–66.1%), and the cost regime and offset are uninformative on this saturated corpus.

Transferability.. We crafted FFE flows against one architecture and scored them with the other three, using the 221 malicious flows that all four detect when clean, so that a transfer is not the target's own false negative; all four

models share the training partition. Flows crafted against XGBoost evade LightGBM in 71.5% of the cases and Random Forest [66] in 55.7%; those crafted against LightGBM, CatBoost and Random Forest evade the non-CatBoost targets in 63.8%–78.7%. CatBoost is the hardest target (24.0% for the XGBoost, LightGBM and Random Forest sources), although attacks against CatBoost itself succeed in only 69.7% of the cases (Supplementary Table S4).

5. Defensive Countermeasures

Standard boosting picks split features by loss reduction alone and may therefore rely on features that an attacker can shift by delaying or padding packets, such as the duration and the rates. We evaluate two defenses: malleability-penalized splitting (MPS), which keeps the model from relying on malleable features, and Lie-guided adversarial training (LGAT), which trains on flows generated along the Lie generators.

5.1. Malleability-penalized tree splitting

Following robust split formulations [53, 55], the split score of feature j is discounted by its physical malleability index

$$M_{\mathrm{mal}}(j) = \mathbb{E}_{x\sim\mathcal{D}_{\mathrm{train}}}\left[\max_{k\in\{1,\dots,4\}} \frac{\delta s_k |\vec{v}_k(x)_j|}{\sigma_j}\right], \tag{21}$$

where σ_j is the feature standard deviation and δs_k the perturbation scale of generator k ($\delta s_1 = 10^4\,\mu$s, $\delta s_2 = 100\,$B, $\delta s_3 = 1\,$pkt, $\delta s_4 = 10^3\,$B). Our experiments evaluate the limit in which the penalty is infinite: the ensemble is trained and queried without the features that temporal dilation changes, namely the duration x_6 and the two rates x_8, x_9 (the two rates also have by

far the largest malleability index). The attacker still perturbs the complete flow and, following the white-box tier, knows the thresholds of the hardened model.

5.2. Lie-guided adversarial training

Standard adversarial training assumes an L_p ball around each input [67], which for network flows does not respect protocol constraints and can generate telemetry that no packet sequence produces [7]. LGAT instead restricts the inner maximization to the physical flow manifold: FFE flows with budget $M = 5$ are generated against the undefended model for a random sample of 5,000 malicious training flows (1.2% of the 416,000 in the training partition, capped to bound the generation cost) and appended to the training set with the malicious label (Supplementary Algorithm S4).

5.3. Evaluation

We compare the undefended XGBoost model (60 trees of depth 6) with MPS, LGAT and MPS+LGAT on the full CSE-CIC-IDS2018 capture. Each model is attacked with FFE at the training budget $M = 5$, with FFE at $M = 20$, and with the surrogate-transfer CAPGD baseline, on the balanced cohort of malicious test flows that the undefended model detects (283 flows, up to 50 per family) restricted to the flows that the evaluated model itself still detects (274, 269 and 274 flows for the hardened models). Table 4 reports Real ASR.

None of the defenses changes clean detection measurably (accuracy 97.28%–97.30%, F1 87.3%, attack recall 77.6%–77.8%). Against the attack that generated the augmentation, LGAT lowers FFE success from 84.1% to 46.5% at

Table 4: Defenses on the full CSE-CIC-IDS2018 capture (XGBoost). Clean accuracy, F1 and attack-class recall on the whole test partition (%); Real ASR (%) of FFE at $M = 5$ and $M = 20$ and of CAPGD on the detected cohort ($N \approx 270$; Wilson 95% half-widths are about 6 points).

Model	**Acc.**	**F1**	**Attack recall**	**FFE** M=5	**FFE** M=20	**CAPGD**
Undefended	97.30	87.31	77.62	84.1	98.9	88.3
MPS	97.30	87.30	77.61	78.8	78.8	71.9
LGAT	97.28	87.26	77.83	46.5	80.3	74.3
MPS + LGAT	97.30	87.30	77.62	62.0	62.0	55.1

$M = 5$, but at $M = 20$ FFE reaches 80.3% against it (undefended 98.9%) and CAPGD 74.3% (undefended 88.3%). MPS alone gives a smaller reduction (78.8% for both FFE budgets, 71.9% for CAPGD). The combination is the least evaded model under the stronger attacks (62.0% at $M = 20$, 55.1% for CAPGD) but is weaker than LGAT at the training budget (62.0% against 46.5%). Differences of a few points, such as 62.0% against 55.1%, are within sampling error for 270 flows and one seed. For the two masked models the $M = 5$ and $M = 20$ rates coincide, so a larger budget does not help FFE once duration and rates are hidden.

No configuration brings evasion below 46%, so none solves the problem. Our reading is that adversarial training with a small budget buys robustness mainly against the attack it was trained on, and that hiding the malleable features, which costs nothing in clean performance, helps less alone but makes the model harder to evade with a larger budget or a surrogate attack when combined with it. These are hypotheses: one seed, one corpus and two attack families, with an attacker that is white-box with respect to the hardened model but not adaptive to its training.

6. Discussion and Conclusions

6.1. What the Results Support, and What They Do Not

Feasibility.. Every flow FFE produced passes the schema check, so "feasible" means feasible with respect to the 12-feature schema. We replayed the first 300 attacked flows per corpus (schema-level success 99.3%, 99.3% and 59.7%) as synthetic packet traces and re-extracted the features (Supplementary Section S8): 99.3%–100% of the evasions persist when chaff packets carry the booked bytes, but only 68.2% on UNSW-NB15 if chaff packets are empty, and no real capture or network stack was involved. A deployed detector may use features outside the twelve (inter-arrival statistics, flag counts) that the four actions also change.

Comparison with the baselines.. FFE exceeds each of the seven baselines on each corpus, significantly so (Table 1), but the ranking is not dominance: a white-box leaf-tuple-style attack that projects every candidate equals it on CSE-CIC-IDS2018, evades slightly more on CIC-IDS2017 and reaches 90.2% against 57.6% on UNSW-NB15 (Supplementary Section S7), so the four generators limit FFE's reach there. It needs 2.8–20 times fewer queries than PSO and MOEVA on the same flows, and its score-only variant without TSBS, which needs no thresholds, still beats every baseline (99.5%, 97.5% and 54.6%; PSO and MOEVA ran on 200 flows). Surrogate-based attacks need far fewer target queries (CAPGD 3.2–16.2 against 14.6–93.7) and come within 10–11 points on the two easier corpora, but fall to 21.1% on UNSW-NB15, where the surrogate agrees least with the target (87.6%), which may explain it. The cost advantage is partly by construction, and in the normalized L_1 distance FFE is not smaller than PSO on two corpora. Boundary

snapping adds at most three points at up to 1.8 times the queries (Section 4.4), so it accounts for little of FFE's success; we did not isolate the Lie structure from a plain greedy search over the same four moves.

Defenses.. Lie-guided adversarial training nearly halves FFE's success at its training budget but protects far less against a larger budget or another attack, and no model brings evasion below 46% (Section 5); the defenses were evaluated on one corpus and one seed, and the masked-feature defense implements the infinite-penalty limit.

Limits of the method.. TSBS needs the split thresholds (Tier 1); without it FFE needs only the score of the target. An attacker who cannot query the target at all depends on transfer, and flows crafted against one architecture evade the others in 24%–79% of the cases on CSE-CIC-IDS2018 (Supplementary Section S5), with CatBoost the hardest target. The packet realization has not been tested on a real network; control is restricted to forward traffic, deep packet inspection is out of scope, and rate dilution lowers the intensity of volumetric floods.

Threats to validity.. The main comparison uses one training seed; the five-seed repetition covers three attacks. Cohorts contain only flows that the models detect when clean (77.6% of the CSE-CIC-IDS2018 test attacks). MOEVA and PSO ran on 200 flows. CAPGD and the NIDSGAN-style generator are our re-implementations with one small surrogate, the baselines use default budgets, and the white-box control is our own greedy re-implementation, not LT-Attack or Cascioli et al., run on one seed. The feasibility check has a 5% tolerance on the coupled rates for all methods. The random split lets

near-duplicates fall on both sides, so accuracies are optimistic.

Each generator has a protocol-compliant packet realization: sender pacing for temporal dilation, padding in protocol fields or ignored application frames (`SSH_MSG_IGNORE`) [60], payload-free pure ACK or keep-alive chaff [58, 62] or zero-length UDP datagrams [59], and handshake window morphing, with sequence numbers and checksums updated via proxy [57, 58, 59].

6.2. Concluding Remarks and Future Directions

This article has measured, at full scale, how vulnerable tree-ensemble NIDS with high benchmark accuracy are to evasion that respects a flow schema. Flow-Field Evasion models four packet-level actions as infinitesimal generators on the flow manifold; trajectories in the continuous Lie cone preserve the conservation invariants and a projection enforces them on the discrete schema. On the full, deduplicated CSE-CIC-IDS2018, CIC-IDS2017 and UNSW-NB15 captures, FFE evades XGBoost in 99.5%, 99.2% and 57.6% of the detected attacks it targets, with schema-feasible flows and 14.6–93.7 target queries per flow. It is significantly more successful than all seven baselines and stays ahead of CAPGD over five retrained seeds, but a white-box tree attack that projects every candidate matches or exceeds it. Evasion depends on the target: LightGBM on CIC-IDS2017 resists FFE because the window target lies in an empty region of the training data (we did not examine the residual failures at an in-support target), and CatBoost resists transfer. Lie-guided adversarial training halves FFE success at the budget it was trained on but protects much less against stronger attacks. Future work includes replay on a real network stack, the official white-box tree attacks, defenses evaluated on all corpora, several seeds and attacks other than the

one used for augmentation, and the remaining public NIDS corpora.

CRediT authorship contribution statement

Quang-Vinh Dang: Conceptualization, Methodology, Software, Formal analysis, Investigation, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research received no external funding.

Data availability

The code, the dataset builders and the per-flow result archives for FFE and all baselines (naive, ZOO-NIDS, HopSkipJump-NIDS, MOEVA, PSO, CAPGD, NIDSGAN-style), the ablations, the transfer study and the defenses are at https://github.com/vinhqdang/adversarial_ids. All flow data come from the public CSE-CIC-IDS2018, CIC-IDS2017 and UNSW-NB15 archives.

References

[1] I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani, Toward generating a new intrusion detection dataset and intrusion traffic characterization, in: Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), 2018, pp. 108–116. doi:10.5220/0006639801080116.

[2] T. Chen, C. Guestrin, XGBoost: A scalable tree boosting system, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794. doi:10.1145/2939672.2939785.

[3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, T.-Y. Liu, LightGBM: A highly efficient gradient boosting decision tree, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 30, 2017, pp. 3146–3154, neurIPS proceedings carry no DOI.
URL https://papers.nips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html

[4] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, A. Gulin, CatBoost: Unbiased boosting with categorical features, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 31, 2018, pp. 6638–6648. doi:10.48550/arXiv.1706.09516.

[5] R. Sommer, V. Paxson, Outside the closed world: On using machine learning for network intrusion detection, in: Proceedings of the 2010 IEEE Symposium on Security and Privacy (S&P), 2010, pp. 305–316. doi:10.1109/SP.2010.25.

[6] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, F. A. Wichmann, Shortcut learning in deep neural networks, Nature Machine Intelligence 2 (11) (2020) 665–673. doi:10.1038/s42256-020-00257-z.

[7] G. Apruzzese, M. Andreolini, L. Ferretti, M. Marchetti, M. Colajanni, Modeling realistic adversarial attacks against network intrusion detection systems, ACM Digital Threats: Research and Practice 3 (3) (2022) 1–19. doi:10.1145/3469659.

[8] F. Pierazzi, F. Pendlebury, J. Cortellazzi, L. Cavallaro, Intriguing properties of adversarial ML attacks in the problem space, in: Proceedings of the 41st IEEE Symposium on Security and Privacy (S&P), 2020, pp. 1332–1349. doi:10.1109/SP40000.2020.00073.

[9] P.-Y. Chen, H. Zhang, Y. Sharma, J. Yi, C.-J. Hsieh, ZOO: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models, in: Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security (AISEC), 2017, pp. 15–26. doi:10.1145/3128572.3140448.

[10] J. Chen, M. I. Jordan, M. J. Wainwright, HopSkipJumpAttack: A query-efficient decision-based attack, in: Proceedings of the 41st IEEE Symposium on Security and Privacy (S&P), 2020, pp. 1277–1294. doi:10.1109/SP40000.2020.00045.

[11] T. Simonetto, S. Ghamizi, M. Cordy, Constrained adaptive attack: Effective adversarial attack against deep neural networks for tabular data, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 37, 2024, pp. 27817–27849. doi:10.52202/079017-0873.

[12] S. Ghamizi, M. Cordy, M. Papadakis, Y. L. Traon, Search-based adversarial testing and improvement of constrained credit scoring systems, in: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), 2020, pp. 1089–1100. doi:10.1145/3368089.3409739.

[13] D. Han, Z. Wang, Y. Zhong, W. Chen, J. Yang, S. Lu, X. Shi, X. Yin, Evaluating and improving adversarial robustness of machine learning-based network intrusion detectors, IEEE Journal on Selected Areas in Communications 39 (8) (2021) 2632–2647. doi:10.1109/JSAC.2021.3087242.

[14] B.-E. Zolbayar, R. Sheatsley, P. McDaniel, M. J. Weisman, S. Zhu, S. Zhu, S. Krishnamurthy, Generating practical adversarial network traffic flows using NIDSGAN, arXiv preprint (2022). arXiv:2203.06694, doi:10.48550/arXiv.2203.06694.

[15] N. Moustafa, J. Slay, UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set), in: Proceedings of the 2015 Military Communications and Information Systems Conference (MilCIS), 2015, pp. 1–6. doi:10.1109/MilCIS.2015.7348942.

[16] Q.-V. Dang, Using machine learning for intrusion detection systems, Computing and Informatics 41 (1) (2022) 12–33. doi:10.31577/cai_2022_1_12.

[17] Q.-V. Dang, Studying the attack detection problem using the dataset CIDDS-001, in: Digital Science (DSIC 2021), Vol. 381 of Lecture Notes in Networks and Systems, Springer, Cham, 2022, pp. 525–532. doi:10.1007/978-3-030-93677-8_46.

[18] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, R. Fergus, Intriguing properties of neural networks, in: Proceedings of the International Conference on Learning Representations (ICLR), 2014. doi:10.48550/arXiv.1312.6199.

[19] I. J. Goodfellow, J. Shlens, C. Szegedy, Explaining and harnessing adversarial examples, in: International Conference on Learning Representations (ICLR), 2015. doi:10.48550/arXiv.1412.6572.

[20] N. Papernot, P. McDaniel, S. Jha, M. Fredrikson, Z. B. Celik, A. Swami, The limitations of deep learning in adversarial settings, in: Proceedings of the 2016 IEEE European Symposium on Security and Privacy (EuroS&P), 2016, pp. 372–387. doi:10.1109/EuroSP.2016.36.

[21] N. Carlini, D. Wagner, Towards evaluating the robustness of neural networks, in: Proceedings of the 38th IEEE Symposium on Security and Privacy (S&P), 2017, pp. 39–57. doi:10.1109/SP.2017.49.

[22] M. Verkerken, L. D'hooge, B. Volckaert, F. D. Turck, G. Apruzzese, "What is the problem space?" Defining host-space adversarial perturbations against network intrusion detection systems, in: Proceedings of the 21st ACM ASIA Conference on Computer and Communications Security (ASIA CCS), 2026, pp. 1043–1059. doi:10.1145/3779208.3807482.

[23] G. Apruzzese, H. S. Anderson, L. Cavallaro, "Real attackers don't compute gradients": Bridging the gap between adversarial ML research and practice in cybersecurity, in: Proceedings of the 1st IEEE Conference on Secure and Trustworthy Machine Learning (SaTML), 2023, pp. 339–364. doi:10.1109/SaTML54575.2023.00031.

[24] K. He, D. D. Kim, M. R. Asghar, Adversarial machine learning for network intrusion detection systems: A comprehensive survey, IEEE Communications Surveys & Tutorials 25 (1) (2023) 538–566. doi:10.1109/COMST.2022.3233793.

[25] J. Vitorino, I. Praça, E. Maia, SoK: Realistic adversarial attacks and defenses for intelligent network intrusion detection, Computers & Security 134 (2023) 103433. doi:10.1016/j.cose.2023.103433.

[26] M. A. Merzouk, F. Cuppens, N. Boulahia-Cuppens, R. Yaich, Investigating the practicality of adversarial evasion attacks on network intrusion detection, Annals of Telecommunications 77 (2022) 763–775. doi:10.1007/s12243-022-00910-1.

[27] R. Sheatsley, N. Papernot, M. Weisman, G. Verma, P. McDaniel, Adversarial examples for network intrusion detection systems, Journal of Computer Security 30 (2022) 727–752. doi:10.3233/JCS-210094.

[28] I. Debicha, B. Cochez, T. Kenaza, T. Debatty, J.-M. Dricot, W. Mees, Adv-Bot: Realistic adversarial botnet attacks against network intrusion detection systems, Computers & Security 129 (2023) 103176. doi:10.1016/j.cose.2023.103176.

[29] I. Debicha, T. Kenaza, I. Charfi, S. Mosbah, M. Sehaki, J.-M. Dricot, Targeted adversarial traffic generation: Black-box approach to evade intrusion detection systems in IoT networks, International Journal of Machine Learning and Cybernetics 17 (2026) 58. doi:10.1007/s13042-025-02873-w.

[30] H. Zhang, D. Han, Y. Liu, Z. Wang, J. Sun, S. Zhuang, J. Liu, J. Dong, Explainable and transferable adversarial attack for ML-based network intrusion detectors, IEEE Transactions on Dependable and Secure Computing 22 (2025) 5090–5107. doi:10.1109/TDSC.2025.3560486.

[31] M. elShehaby, A. Matrawy, A novel perturb-ability score to mitigate evasion adversarial attacks on flow-based ML-NIDS, Journal of Information Security and Applications 99 (2026) 104409. doi:10.1016/j.jisa.2026.104409.

[32] Z. Lin, Y. Shi, Z. Xue, IDSGAN: Generative adversarial networks for attack generation against intrusion detection, in: Advances in Knowledge Discovery and Data Mining (PAKDD), Lecture Notes in Computer Science, 2022, pp. 79–91. doi:10.1007/978-3-031-05981-0_7.

[33] K. Kireev, B. Kulynych, C. Troncoso, Adversarial robustness for tabular data through cost and utility awareness, in: Proceedings of the Network and Distributed System Security Symposium (NDSS), 2023. doi:10.14722/ndss.2023.24924.

[34] T. Simonetto, S. Ghamizi, M. Cordy, TabularBench: Benchmarking adversarial robustness for tabular deep learning in real-world use-cases, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 37, 2024, pp. 78394–78430. doi:10.52202/079017-2492.

[35] Z. He, C. Ouyang, L. Wen, C. Liu, C. Moreira, TabAttackBench: A benchmark for adversarial attacks on tabular data, Expert Systems with Applications 301 (2026) 130491. doi:10.1016/j.eswa.2025.130491.

[36] S. Dyrmishi, M. Djilani, T. Simonetto, S. Ghamizi, M. Cordy, Insights on adversarial attacks for tabular machine learning via a systematic literature review, arXiv preprint (2025). arXiv:2506.15506, doi:10.48550/arXiv.2506.15506.

[37] J. Vitorino, M. Silva, E. Maia, I. Praça, An adversarial robustness benchmark for enterprise network intrusion detection, in: Foundations and Practice of Security (FPS 2023), Lecture Notes in Computer Science, 2024, pp. 3–17. doi:10.1007/978-3-031-57537-2_1.

[38] L. Cascioli, L. Devos, O. Kuželka, J. Davis, Faster repeated evasion attacks in tree ensembles, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 37, 2024, pp. 24762–24794. doi:10.52202/079017-0780.

[39] L. Devos, L. Perini, W. Meert, J. Davis, Detecting evasion attacks in deployed tree ensembles, in: Machine Learning and Knowledge Discovery in Databases: Research Track (ECML PKDD 2023), Lecture Notes in Computer Science, 2023, pp. 120–136. doi:10.1007/978-3-031-43424-2_8.

[40] D. Vos, S. Verwer, Efficient training of robust decision trees against adversarial examples, in: Proceedings of the 38th International Conference on Machine Learning (ICML), Vol. 139 of Proceedings of Machine Learning Research, 2021, pp. 10586–10595. doi:10.48550/arXiv.2012.10438.

[41] D. Vos, S. Verwer, Robust optimal classification trees against adversarial examples, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 36, 2022, pp. 8520–8528. doi:10.1609/aaai.v36i8.20829.

[42] S. Calzavara, L. Cazzaro, G. E. Pibiri, N. Prezza, Verifiable learning for robust tree ensembles, in: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS), 2023, pp. 1850–1864. doi:10.1145/3576915.3623100.

[43] S. Calzavara, L. Cazzaro, C. Lucchese, G. E. Pibiri, Verifiable boosted tree ensembles, in: 2025 IEEE Symposium on Security and Privacy (SP), 2025, pp. 2057–2075. doi:10.1109/SP61157.2025.00022.

[44] B. Lindqvist, On the adversarial robustness of decision trees and a symmetry defense, IEEE Access 13 (2025) 16120–16132. doi:10.1109/ACCESS.2025.3530695.

[45] G. Engelen, V. Rimmer, W. Joosen, Troubleshooting an intrusion detection dataset: the CICIDS2017 case study, in: 2021 IEEE Security and Privacy Workshops (SPW), 2021, pp. 7–12. doi:10.1109/SPW53761.2021.00009.

[46] M. Lanvin, P.-F. Gimenez, Y. Han, F. Majorczyk, L. Mé, E. Totel, Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes, in: Risks and Security of Internet and Systems (CRiSIS 2022), Lecture Notes in Computer Science, 2023, pp. 18–33. doi:10.1007/978-3-031-31108-6_2.

[47] L. Liu, G. Engelen, T. Lynar, D. Essam, W. Joosen, Error prevalence in NIDS datasets: A case study on CIC-IDS-2017 and CSE-CIC-IDS-2018, in: 2022 IEEE Conference on Communications and Network Security (CNS), 2022, pp. 254–262. doi:10.1109/CNS56114.2022.9947235.

[48] M. Cantone, C. Marrocco, A. Bria, Machine learning in network intrusion detection: A cross-dataset generalization study, IEEE Access 12 (2024) 144489–144508. doi:10.1109/ACCESS.2024.3472907.

[49] A. Kantchelian, J. D. Tygar, A. Joseph, Evasion and hardening of tree ensemble classifiers, in: Proceedings of the 33rd International Conference on Machine Learning (ICML), 2016, pp. 2387–2396. doi:10.48550/arXiv.1509.07892.

[50] A. Ignatiev, Y. Izza, P. J. Stuckey, J. Marques-Silva, Using MaxSAT for efficient explanations of tree ensembles, in: Proceedings of the 36th AAAI Conference on Artificial Intelligence, Vol. 36, 2022, pp. 3776–3785. doi:10.1609/aaai.v36i4.20292.

[51] Y. Izza, A. Ignatiev, J. Marques-Silva, On tackling explanation redundancy in decision trees, Journal of Artificial Intelligence Research 75 (2022) 261–321. doi:10.1613/jair.1.13575.

[52] C. Zhang, H. Zhang, C.-J. Hsieh, An efficient adversarial attack for tree ensembles, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, 2020, pp. 16165–16176. doi:10.48550/arXiv.2010.11598.

[53] H. Chen, H. Zhang, S. Si, Y. Li, D. Boning, C.-J. Hsieh, Robust decision trees against adversarial examples, in: Proceedings of the 36th International Conference on Machine Learning (ICML), 2019, pp. 1122–1131. doi:10.48550/arXiv.1902.10660.

[54] M. Andriushchenko, M. Hein, Provably robust boosted decision stumps and trees against adversarial attacks, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 32, 2019, pp. 12997–13008. doi:10.48550/arXiv.1906.03526.

[55] S. Calzavara, C. Lucchese, G. Tolomei, S. A. Abebe, S. Orlando, Treant: training evasion-aware decision trees, Data Mining and Knowledge Discovery 34 (5) (2020) 1390–1420. doi:10.1007/s10618-020-00694-9.

[56] M. J. Hashemi, E. Keller, Enhancing robustness against adversarial examples in network intrusion detection systems, in: Proceedings of the 2020 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN), 2020, pp. 37–43. doi:10.1109/NFV-SDN50289.2020.9289869.

[57] J. Postel, Internet protocol, RFC 791, RFC Editor (1981). doi:10.17487/RFC0791.

[58] W. Eddy, Transmission control protocol (TCP) specification, RFC 9293, RFC Editor (2022). doi:10.17487/RFC9293.

[59] J. Postel, User datagram protocol, RFC 768, RFC Editor (1980). doi:10.17487/RFC0768.

[60] T. Ylonen, C. Lonvick, The secure shell (SSH) transport layer protocol, RFC 4253, RFC Editor (2006). doi:10.17487/RFC4253.

[61] M. Allman, V. Paxson, E. Blanton, TCP congestion control, RFC 5681, RFC Editor (2009). doi:10.17487/RFC5681.

[62] D. Borman, B. Braden, V. Jacobson, R. Scheffenegger, TCP extensions for high performance, RFC 7323, RFC Editor (2014). doi:10.17487/RFC7323.

[63] C. Carathéodory, Vorlesungen über reelle Funktionen, B. G. Teubner, Leipzig, 1918.

[64] M. Nagumo, Über die Lage der Integralkurven gewöhnlicher Differentialgleichungen, Proceedings of the Physico-Mathematical Society of Japan. 3rd Series 24 (1942) 551–559. doi:10.11429/ppmsj1919.24.0_551.

[65] H. Brézis, On a characterization of flow-invariant sets, Communications on Pure and Applied Mathematics 23 (2) (1970) 261–263. doi:10.1002/cpa.3160230211.

[66] L. Breiman, Random forests, Machine Learning 45 (1) (2001) 5–32. doi:10.1023/A:1010933404324.

[67] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, A. Vladu, Towards deep learning models resistant to adversarial attacks, in: International Conference on Learning Representations (ICLR), 2018. doi:10.48550/arXiv.1706.06083.